

Notes Go

Suite des variables

- Il n'y a pas de changement de type possible, une fois qu'une variable est déclarée, elle ne peut pas changer de type.
- Les variables ne peuvent pas être undefined ou null, elles ont une valeur par défaut (0 pour les int, "" pour les string, etc.)
- Il y a trois manières de déclarer une variable :
 - `var x int = 10`
 - `var x = 10` (type inféré)
 - `x := 10` (type inféré)
 - Il est possible de déclarer plusieurs variables en même temps :
 - `var x, y int = 10, 20`
 - `x, y := 10, 20`
- On peut échanger les valeurs de deux variables sans utiliser de variable temporaire :
 - `x, y = y, x`
- Go refuse les variables non utilisées, ce qui est une bonne pratique pour éviter les bugs liés à des variables oubliées.

Les fonctions

- Les fonctions ne peuvent pas être surchargées, c'est-à-dire qu'on ne peut pas avoir deux fonctions avec le même nom mais des paramètres différents.
- Les fonctions peuvent retourner plusieurs valeurs, ce qui est très pratique pour retourner à la fois un résultat et une erreur :
 - `func divide(a, b float64) (float64, error) { ... }`
- Le `_` permet d'ignorer une valeur de retour :
 - `result, _ := divide(10, 2)`
- Le naked return permet de retourner les variables nommées sans les spécifier dans le return :
 - `return` au lieu de `return result, err`
- (erreur comme valeur) est plus explicite que les exceptions. On voit visuellement dans le code qu'une opération peut échouer.
- Les fonctions variadiques permettent de passer un nombre variable d'arguments :
 - `func sum(nums ...int) int { ... }`
- closure : une fonction qui peut accéder aux variables de son environnement même après que la fonction parente ait terminé son exécution.
- La boucle for est la seule boucle en Go, elle peut être utilisée de différentes manières :
 - `for i := 0; i < 10; i++ { ... }`
 - `for i, v := range slice { ... }`
 - `for { ... }` (boucle infinie)

Pas d'héritage en Go → composition par embedding (Lego!) Struct tags → `json: 'nom,omitempty'` — `json: '-'`
pour ne pas sérialiser Go gère auto `&` et `*` pour les méthodes → `p.MethodePtr() == (&p).MethodePtr()`

Les slices, maps, pointeurs et structs

- Les slices sont des vues dynamiques sur des tableaux, elles peuvent être redimensionnées avec `append`, copiées avec `copy` et parcourues avec `range`.
- Les maps sont des collections de paires clé-valeur, elles sont très utiles pour stocker des données associatives. L'idiome pour vérifier si une clé existe est `val, ok := m[k]`.
- Les pointeurs permettent de manipuler directement la mémoire, ils sont utilisés pour passer des références à des fonctions ou pour modifier des structs. L'opérateur `&` permet d'obtenir l'adresse d'une variable, tandis que l'opérateur `*` permet de déréférencer un pointeur pour accéder à la valeur qu'il pointe.
- Les structs sont des types de données composés qui peuvent contenir plusieurs champs. Ils sont souvent utilisés pour représenter des objets ou des entités du monde réel. Les méthodes peuvent être associées à des structs pour définir des comportements, et l'embedding permet de composer des structs pour réutiliser du code sans avoir besoin d'héritage.

Visibilité

- Privé : commence par une lettre minuscule, accessible uniquement dans le même package.
- Public : commence par une lettre majuscule, accessible depuis n'importe quel package.
- Il n'y a pas de mot-clé `private` ou `public` en Go, c'est la convention de nommage qui détermine la visibilité.

Defer

- `defer` permet de différer l'exécution d'une fonction jusqu'à la fin de la fonction courante, même en cas de panic.